

AULA 6

Interfaces próprias e detecção inteligente

Etapa 3 · terça, 25/08/2026 · sala SJ205 · Prof. Dácio Moreira de Souza



**O TP1 vence nesta sexta, 28/08.
A primeira hora de hoje é clínica.**

Conteúdo novo depois — e ele é do TP2, não do TP1.

Onde o seu TP1 está, agora

- G1.0 ambiente · G1.1 projeto declarado · G1.2 grafo de imagem no ar
- G1.3 segmentação detectando o SEU objeto · G1.4 serviço + rqt_graph (venceu ontem)
- G1.5 relatório fechado — vence amanhã, 26/08
- G1.6 entrega: branch entrega-tp1, tag tp1, ZIP + PDF no Moodle — sexta
- Marque [x], [] ou [!]. Um [!] declarado hoje custa menos que um [!] descoberto na sexta

Diagnóstico em um comando

```
cd ~/SEU-REPO
bash recursos/check-ambiente.sh # G1.0
grep -c "PB:PROJETO\|PB:TRILHA" PROJETO.md # G1.1
ros2 topic hz /vision/contagem # G1.2 e G1.3
ros2 service call /vision/status \
    std_srvs/srv/Trigger "{}" # G1.4
ls docs/evidencias/tp1/ # a prova de tudo
```

Evidência que ficou no terminal não existe — o terminal fecha.

O relatório é um documento de decisões

- Domínio e derivação — que problema, de quem, e por que este
- Decisões técnicas — o que você escolheu e o que descartou
- Experimento de oclusão — o que aconteceu e o que implica no seu domínio
- Limitações conhecidas — o que o sistema NÃO faz, dito por você primeiro
- Uso de IA — o que foi gerado, revisado, e o que você sabe explicar



Se o avaliador não consegue abrir, não existe.

Teste cada link em janela anônima. Você está logado em tudo.

Baixar o material desta aula

```
# 1) baixar o material (pode repetir sempre)
rm -rf /tmp/PBRoboticos_prof_dacio
cd /tmp && git clone --depth 1 \
    https://github.com/Prof-Dacio-INFNET/PBRoboticos_prof_dacio.git

# 2) copiar os DOIS pacotes para dentro do SEU projeto
cp -r /tmp/PBRoboticos_prof_dacio/exemplos/aula06-interfaces/pb_interfaces \
    /tmp/PBRoboticos_prof_dacio/exemplos/aula06-interfaces/aula06_percepcao \
    ~/projeto-pb-SEU-USUARIO/ros2_ws/src/
```

Você não trabalha dentro do repositório da disciplina: copia o pacote para o SEU projeto e compila lá.

ETAPA 3

Quando a mensagem pronta acaba

Três. Três o quê?

```
$ ros2 topic echo /vision/contagem --once  
data: 3
```

A informação existe dentro do nó — área, posição, contorno — e é jogada fora na publicação, porque o tipo não tem onde guardar.

Quando criar interface própria

NÃO vale

- um número solto → `Int32` continua certo
- um texto → `String` resolve
- "por organização", sem uso concreto
- JSON dentro de `String`: perde tipo, introspecção e C++

VALE

- três ou mais campos que andam sempre juntos
- dois tópicos que só fazem sentido lidos juntos
- vocabulário do domínio que ninguém mais tem
- contrato que sobrevive à troca do miolo

Por que o pacote de interfaces é SEPARADO

- Gerar interface é gerar código: `.msg` → C++, Python, headers
- Quem gera é o `roscpp`, e ele roda em pacote `ament_cmake`
- Um pacote `ament_python` — o dos seus nós — não tem essa maquinaria
- É assim que o `sensor_msgs` chega até você: só interfaces, sem nós de terceiros
- Separar permite que outros dependam das suas mensagens sem compilar os seus nós

A árvore que você vai repetir a vida toda

```
ros2_ws/src/  
  pb_interfaces/          # ament_cmake -- S0 .msg e .srv  
    msg/Deteccao.msg  
    msg/Deteccoes.msg  
    srv/StatusVisao.srv  
    CMakeLists.txt  
    package.xml  
  
  aula06_percepcao/      # ament_python -- os nos  
    aula06_percepcao/detector.py  
    setup.py    package.xml
```

Tentar gerar interface no pacote Python dá erro de CMake num lugar onde você nem sabia que havia CMake.

As três linhas que fazem o pacote gerar código

```
<buildtool_depend>roscpp</buildtool_depend>  
<exec_depend>roscpp</exec_depend>  
<member_of_group>roscpp</member_of_group>
```

Sem elas o pacote compila e não gera nada — o pior tipo de erro, o silencioso.

Anatomia de um .msg

```
# Deteccao.msg -- uma deteccao individual
string  classe          # rotulo do SEU dominio
float32  confianca       # 0.0 a 1.0
int32    x               # centro, em pixels
int32    y
int32    area

# Deteccoes.msg -- o que se viu em UM quadro
std_msgs/Header header
Deteccao[]  deteccoes    # array de tamanho variavel
uint32      total
```

O Header não é enfeito.

Sem stamp você não sabe se a detecção é de agora ou de três segundos atrás — a diferença entre um robô que reage e um que alucina.

O mesmo serviço, outro vocabulário

TP1 — std_srvs/srv/Trigger

- success: bool
- message: string
- → diz "estou vivo"

Agora — StatusVisao.srv

- ativo: bool
- classe_alvo: string
- total_detectado: uint32
- confianca_media: float32
- → diz O QUE se está vendo

Compilar, inspecionar, consumir

```
# a ORDEM importa: interface antes do no que a importa
colcon build --packages-select pb_interfaces
source install/setup.bash
colcon build --packages-select aula06_percepcao --symlink-install
source install/setup.bash

ros2 interface show pb_interfaces/msg/Deteccoes
ros2 launch aula06_percepcao percepcao.launch.py classe:=tomate_maduro
ros2 service call /vision/status pb_interfaces/srv/StatusVisao "{}"
```

Mudou um .msg? Recompila E dê source de novo, em TODOS os terminais abertos.

ETAPA 4, A CAMINHO

Onde a cor deixa de bastar

Segmentar não é detectar

- Cor responde "onde há vermelho". Detectar responde "onde há um tomate — e o quanto eu acredito"
- A confiança do exemplo é a SOLIDEZ do contorno: um substituto honesto, não uma probabilidade
- Na oclusão, a contagem cai de 2 para 1 e a confiança cai junto — o detector sinaliza a própria dúvida
- Um modelo treinado dá esse sinal de forma direta e calibrada — e exige dados, rótulos e avaliação
- O que NÃO muda é a interface: classe, confiança e posição servem para os dois



Trocar o miolo sem trocar o contrato.

É por isso que esta aula vem antes daquela.

Antes da próxima terça

- Até sexta 28/08: entregar o TP1 — checklist completo no site
- Tarefa da Aula 6: criar o seu <projeto>_interfaces e migrar o /vision/status
- A tarefa é do TP2 e pode esperar o fim de semana. A entrega, não
- Aula 7, 01/09: Etapa 4 — SLAM e percepção veicular